

# Vehicle Selector Pro

A Production-Grade Shopify App for Vehicle Fitment

---

CLIENT DELIVERABLE · AUGUST 2026

**Live demo:** <https://vehicle-selector-pro.fly.dev/demo>

**Merchant admin:** <https://vehicle-selector-pro.fly.dev/demo/admin>

**Repository:** <https://github.com/ai-dev-2024/vehicle-selector-pro>



# Aftermarket parts sell badly online

---

- A brake pad or wiper blade only fits an exact Year / Make / Model / Trim / Engine (YMMTE) — compatibility is the whole sale.
- Wrong-fit means wrong parts, angry customers, and costly returns. Merchants either bury the data in dense tables or hard-code rules per product.
- Commercial fitment feeds lock merchants into contracts and ongoing per-part fees.
- Verdict: fitment data is kept in silos, rots quickly, and never pays for itself.

## THE SOLUTION

# One dataset, synced everywhere

---

- Fitments are entered once into a normalized multi-tenant cache, then pushed to Shopify metafields automatically.
- Customers pick Year → Make → Model → Trim → Engine behind an HMAC-signed App Proxy and only ever see parts that fit.
- Product pages render their own Guaranteed Exact / Does NOT Fit / Universal Fit badge.
- Merchants manage everything from a Polaris admin — no per-part fees, no brittle scripts, no vendor lock-in.

03

## WHY IT WORKS

# A real, browsable demo catalog

---

|                                   |                                 |                              |                                |
|-----------------------------------|---------------------------------|------------------------------|--------------------------------|
| <b>48</b><br>YMMTE configurations | <b>204</b><br>verified fitments | <b>40</b><br>mapped products | <b>8</b><br>brands represented |
|-----------------------------------|---------------------------------|------------------------------|--------------------------------|

Try it live — no setup required. Every number above is visible in the running app.

04

## HOW IT WORKS

# A cascading fitment engine

---

- Shopper selects Year → Make → Model → Trim → Engine; each request is HMAC-verified by the App Proxy.
- The local PostgreSQL cache answers instantly — the storefront never waits on Shopify.
- Synced fitments are written to `custom.vehicle_fitment` via GraphQL `metafieldsSet` in batches of 25.
- Product pages, collection filters, and the My Garage feature all stay perfectly in sync.

05

## FEATURES

# Built to convert and to administer

---

- Exact / No-fit / Universal badges

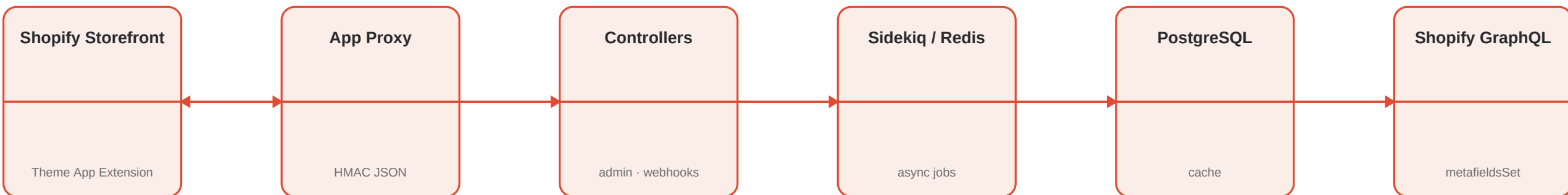
- My Garage (saved vehicles)

- Mobile-f

06

# Three clean layers, zero ScriptTag

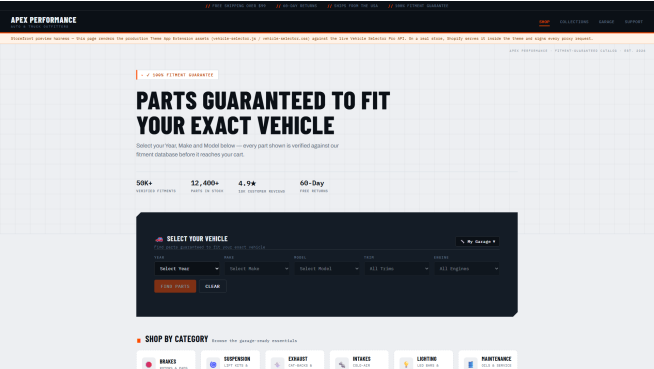
---



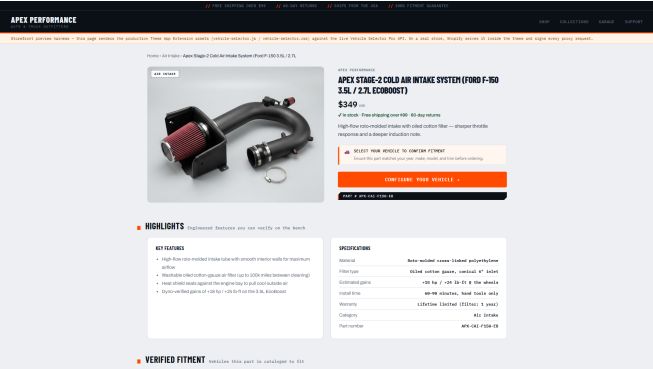
Zero ScriptTag API — pure Theme App Extension, signed App Proxy, and safe webhooks.



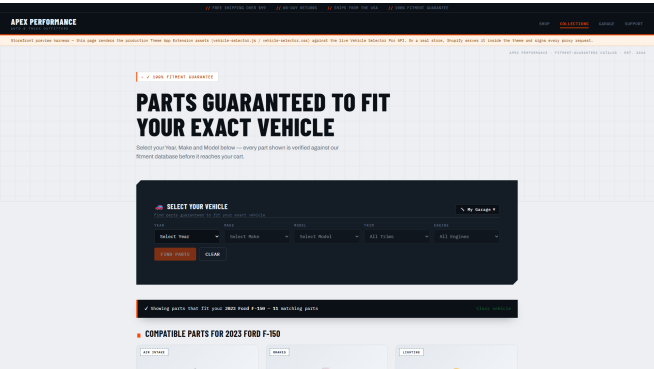
# What shoppers see



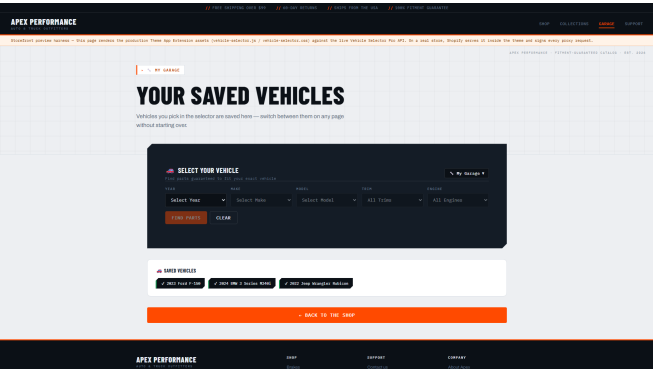
Cascading selector



Fitment badge on PDP

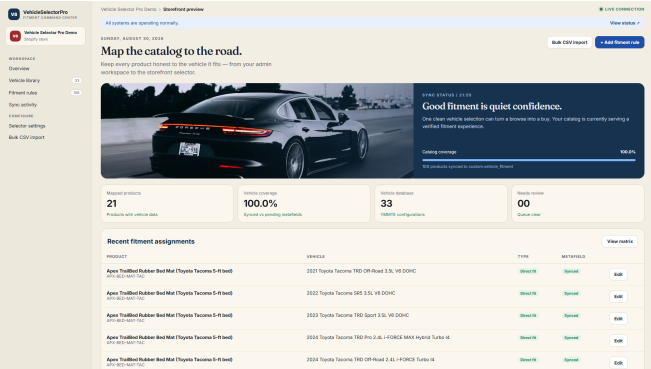


Filtered collection

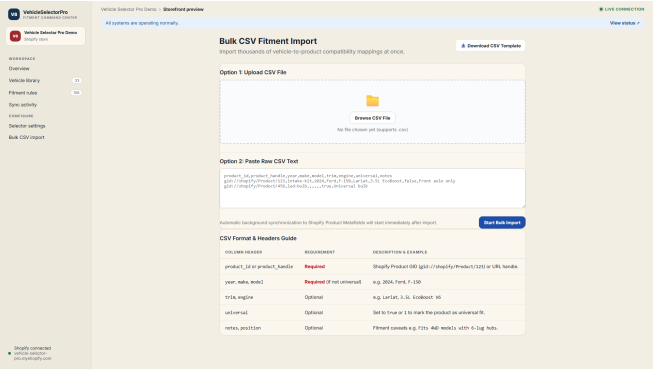


My Garage

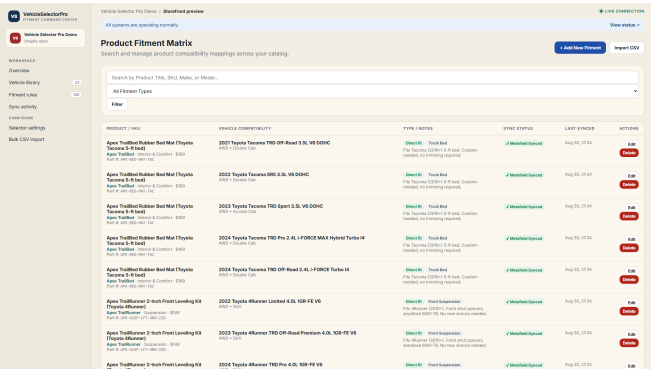
# What merchants control



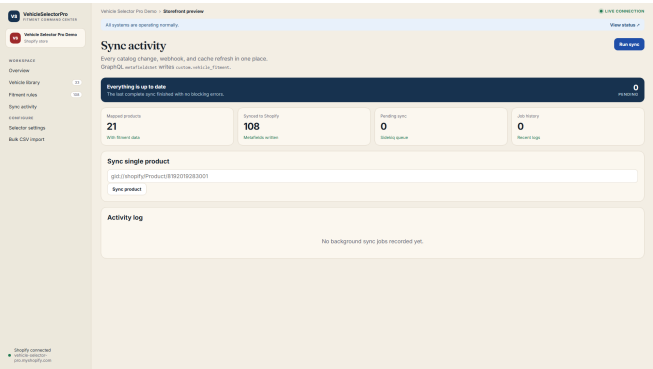
Command center



Bulk import



Fitment matrix



Sync monitor

# Safe, multi-tenant, App-Store ready

---

- Standard OAuth 2.0 with encrypted tokens and per-shop data isolation.
- HMAC-SHA256 verification on every App Proxy and webhook request (constant-time).
- rack-attack throttling and CSP frame-ancestors locked to myshopify.com.
- Full GDPR coverage: data-request, redact, and shop/redact handlers plus a public privacy policy.

# Engineering-grade stack

---

• PostgreSQL cache · SQLite dev

• Polaris · ERB

# Deploy & install on your own store

---

- Create the app in Shopify Partners — copy the API key & secret, set the app and redirect URLs.
- Deploy the Rails app to Fly.io / Render with PostgreSQL, Redis and Sidekiq; boot with rails db:prepare.
- Register the App Proxy subpath /apps/vehicle-selector, then install on the store via OAuth (webhooks auto-register).
- Create the one-time custom.vehicle\_fitment metafield definition in Admin (Product, JSON, Public read).
- Drop in the Theme App Extension blocks (vehicle selector + fitment badge) and publish the theme.
- Bulk-import fitments, run the sync, verify PDP badges & filters, and go live.

## ROADMAP

# A foundation built to grow

---

- Shopify Billing API for recurring and one-time plans.
- Supersession & OE-number matching for a richer part engine.
- Fitment confidence scoring and smarter universal/exact priority.
- White-label theming, translations, and merchant analytics.

# Built at the ceiling of a 2018 ultrabook

---

- The entire project — Rails app, Shopify integration, storefront, diagrams, this deck, and both demo videos — was produced on one machine.
- ASUS ZenBook UX433FA · Intel Core i7-8565U @ 1.80 GHz · 16 GB RAM.
- Intel UHD 620 integrated graphics only — no GPU, no cluster; every frame of both demo videos was rendered on the CPU.
- The output was capped by tooling, not ambition. Given a real workstation, a GPU, and paid frontier models, the same foundation leaps a level.

# Free tools, minimal hardware — maximum headroom

---

No paid AI models, APIs, or commercial tooling were used anywhere — and the whole result was built and rendered on a 15-watt 2018 ultrabook with no GPU. Both are deliberate. They prove what free tooling on minimal local compute already achieves, and set up a fast leap forward: with proper hardware and paid frontier models, the ceiling lifts dramatically.



SEE IT LIVE

# Watch the three narrated walkthroughs

---

- Live application — [watch the real deployment](#) · 1:27
- Merchant command center — [watch](#) · 2:09
- Shopper experience — [watch](#) · 2:21

READY WHEN YOU ARE

# Let's take the next step together

---

- › **Try the live demo** — <https://vehicle-selector-pro.fly.dev/demo>
- › **Explore the merchant admin** — <https://vehicle-selector-pro.fly.dev/demo/admin>
- › **Open the repository** — <https://github.com/ai-dev-2024/vehicle-selector-pro>

Both the live storefront and merchant admin are running right now.